Write Verilog and Testbench Code to Design and Simulation of a 4-Bit Johnson Counter

Aim

To design, implement, and simulate a **4-bit Johnson Counter** using Verilog HDL and verify its unique circulating state transitions using a testbench in Quartus Prime.

Objective:

- Understand the operation and characteristics of shift register-based counters.
- Study the design mechanics of a modified ring counter (Johnson Counter).
- Write robust Verilog HDL code for a sequential Johnson structure.
- Develop an edge-triggered clock testbench for structural functional verification.
- Simulate and verify the cyclic sequence, synchronous state transitions, and reset behavior

Theory:

A **Johnson counter** (also recognized as a twisted ring counter, switch-tail counter, or walking counter) is a modified synchronous shift register layout where the inverted output of the last flip-flop is cross-fed directly back into the serial input of the first flip-flop.

Unlike a standard ring counter that requires preset initialization data to circulate a single high bit, a Johnson counter starts cleanly from an all-zero state. A **4-bit Johnson counter** utilizes **4 flip-flops** and naturally steps through 8 **distinct states** ($2 \times n = 2 \times 4 = 8$). This layout makes it highly efficient, providing twice as many distinct operational states as a traditional ring counter using the exact same number of registers.

Working

The 3-bit Johnson counter updates its state vector on every rising edge of the clk signal. When the active-low reset line drops to 0, the counter resets symmetrically, clearing all registers instantly to 4'b0000.

When reset returns to 1, the serial input for the first bit element becomes the inverse of the final bit element (~sim Q [3]). This inverted value propagates down the internal shift line on subsequent active clock transitions.

Counting Sequence

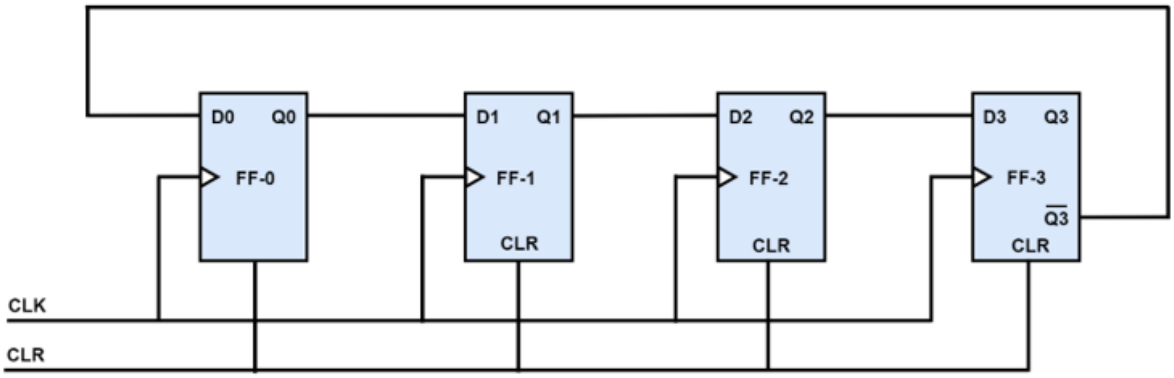
0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → (Repeats back to 0000)

Note: Out of 8 total possible structural configurations in a 4-bit space, 2 states (0010 and 0101) represent unused/invalid operational states. If the system drifts into an invalid state due to power noise, a well-implemented synchronous structure is designed to return to the valid track.

The basic structural progression rule satisfies:

$$Q_{next} = \{\sim Q[2], Q[1], Q[0]\}$$

Block Diagram:-



Operation Table

Present State (Q_3 Q_2 Q_1 Q_0)	Next State (Q_3^+ Q_2^+ Q_1^+ Q_0^+)	Shift Operation Description
0000	0001	Inverted 0 from MSB (Q_3) feeds into LSB (Q_0)
0001	0011	Inverted 0 from MSB feeds into LSB; bits shift left
0011	0111	Inverted 0 from MSB feeds into LSB; bits shift left
0111	1111	Inverted 0 from MSB feeds into LSB; bits shift left
1111	1110	Inverted 1 from MSB feeds into LSB; bits shift left
1110	1100	Inverted 1 from MSB feeds into LSB; bits shift left
1100	1000	Inverted 1 from MSB feeds into LSB; bits shift left
1000	0000	Inverted 1 from MSB feeds into LSB; cycles back

Algorithm

- Start the program.
- Define clk and reset as control inputs.
- Define a 3-bit output vector register Q to track the counter states.
- Establish an edge-triggered process block sensitive to the rising edge of the clock (posedge clk) or the falling edge of the reset signal (negedge reset).
- Evaluate reset: If active-low (0), force the state vector Q immediately to 3'b000.
- If reset is inactive, execute a right shift where the new MSB ($Q[2]$) takes the value of the inverted LSB ($\sim Q[0]$).
- Apply sequential input stimuli and cycle the clock within the testbench framework.
- Verify output waveform transitions against the expected 6-state operation table.
- Stop.

Verilog Code (Design Module)

```
module johnson_counter_4bit (
   input clk,
   input reset,
   output reg [3:0] Q
);

always @(posedge clk or psedge reset) begin
   if (reset) begin
      Q <= 4'b0000;
   end else begin
      // Shift right and feed the inverted LSB (Q[0]) back into the MSB (Q[2])
      Q [3] <= Q [2];
      Q [2] <= Q [1];
      Q [1] <= Q [0];
      Q [0] <= {~Q[3]; // Inverted feedback from last stage

   end
end

Assign out = Q;

endmodule
```

Testbench

```
module johnson_counter_3bit_tb;
   reg clk;
   reg reset;
   wire [3:0] Q;

   // Instantiate Device Under Test (DUT)
   johnson_counter_4bit DUT (
      .clk(clk),
      .reset(reset),
      .Q(Q)
   );

   // Generation of Clock signal (Period = 10ns)
   always #5 clk = ~clk;

   initial begin
      // Initialize Inputs
      clk = 0;
      reset = 1; // Apply active- high reset
      #12;

      // Release reset to launch the Johnson cycle
      reset = 0;
      #100; // Allow ample time to watch multiple full 8-state cycles

      $stop; // End Simulation
   end
endmodule
```

Simulation Procedure (Quartus Prime & Questa Sim)

1. Open Intel Quartus Prime and launch the **New Project Wizard**.
2. Name your working directory and append both files: johnson_counter_4bit.v and johnson_counter_4bit_tb.v.
3. Mark johnson_counter_4bit as the **Top-Level Design Entity** of your design.
4. Run synthesis compilation via **Processing → Start Compilation** and check that there are no compilation errors.
5. Launch your simulator (Questa Sim / ModelSim) and compile your Verilog design files.
6. Designate johnson_counter_4bit_tb as your explicit simulation top-level module.
7. Append clk, reset, and the output vector Q directly onto the Waveform display window.
8. Run the simulation for at least **150 ns** to capture a complete sequence.
9. Analyze wave patterns and compare the hex/binary states against your target tracking matrix.

Expected Simulation Result

Reset (reset)	Counter Output Vector ($Q[3:0]$)	Operation Observed
0	0000	Asynchronous Reset Forced Clean
1	0001	State 1: Inverted MSB (1) shifted into LSB
1	0011	State 2: Progressing walking pattern upward
1	0111	State 3: Shifting bits to the left
1	1111	State 4: All registers filled with ones
1	1110	State 5: Inverted MSB (0) shifts back inside LSB
1	1100	State 6: Progressing zero clearing pattern from right
1	1000	State 7: Only one high bit left at the MSB
1	0000	State 8: Returns to initial state (Full Loop Complete)

Observation

The 4-bit Left-Shifting Johnson Counter was designed and verified using Verilog HDL. Through the testbench implementation, the outputs were observed across continuous clock periods. The counter safely navigated its 8 valid operational states synchronously on every positive clock edge without glitches or cascading state delays, matching the targeted shift sequence.

Applications

- Multi-phase clock generator circuits inside microprocessors.
- Digital frequency dividers (divides the input clock rate by 2n = 8).
- Hardware design pattern generation for digital communication networks.
- Low-noise instrumentation systems that require minimal state-transition hazards.

Advantages

- Does not require initialization logic or a single-bit seed to operate, unlike a standard ring counter.
- Fully synchronous layout eliminates dangerous glitches and racing hazards.
- Offers double the operational states (2n) of a ring counter using an identical number of flip-flops.
- Simple decoding logic since any state can be identified using a basic 2-input gate

Conclusion

The 4-bit Left-Shifting Johnson Counter was designed and verified using Verilog HDL. Through the testbench implementation, the outputs were observed under synchronous execution conditions. The counter updated states simultaneously at every positive edge of the clock signal, confirming its synchronous behavior and eliminating cumulative propagation delays. The simulation waveforms successfully matched the theoretical state transitions, confirming a total loop length of 8 states and proving the functional correctness of the digital design.

Viva Questions

1. How does changing a Johnson counter from right-shifting to left-shifting impact its state transitions?
2. What is the output frequency of a 4-bit Johnson counter if the input clock frequency is 80 MHz?
3. What are "unused" or "invalid" states in a counter design, and what happens if a Johnson counter accidentally enters one?
4. How can you modify a Johnson counter design so that it automatically self-corrects from an illegal state?
5. Why are Johnson counters favoured over binary counters in high-speed, low-power digital designs?